

1. Considere o seguinte tipo de dados utilizado para representar um nó de uma lista ligada de mensagens:

```
typedef struct message {  
    int value;  
    int timestamp;  
    struct message * next;  
} *LLig;
```

Cada nó representa um registo de mensagem contendo:

- *value*: o identificador da mensagem
- *timestamp*: um identificador temporal associado à mensagem
- *next*: um apontador para o próximo nó na lista ligada

Um servidor de mensagens registou uma sequência de mensagens. Cada registo contém um identificador de mensagem (*value*) e um identificador temporal (*timestamp*). Devido a um erro de rede, algumas mensagens foram enviadas várias vezes.

A sequência de mensagens é armazenada como uma lista ligada simples composta por nós do tipo *LLig*.

Para limpar os registos, o sistema percorre a lista ligada e aplica o seguinte algoritmo:

Para cada valor de mensagem (*value*), podem existir vários registos na lista. Quando existem várias ocorrências do mesmo *value*, considera-se que correspondem a mesma mensagem enviada várias vezes. Nesse caso, mantém-se apenas o nó com maior *timestamp* (a mensagem mais recente) e ignoram-se os restantes.

Após este processo, cada valor aparece apenas uma vez na lista resultante. Por fim, o sistema calcula a soma dos *timestamps* das mensagens que permanecem. Por exemplo, considere a seguinte lista

(7,100) -> (3,105) -> (8,120) -> (3,110) -> NULL

Existem duas ocorrências da mensagem 3, pelo que o algoritmo é aplicado:

(7,100) -> (8,120) -> (3,110) -> NULL

A soma dos 'timestamp' neste caso é:

100 + 120 + 110 = 330

Considere a seguinte lista de mensagens registada pelo servidor:

**(7,17) -> (9,15) -> (1,18) -> (4,31) -> (2,33) -> (7,31) -> (7,36) ->
(2,5) -> (5,30) -> (2,34) -> (3,15) -> (1,30) -> (2,40) -> (5,36) ->
(5,10) -> (4,7) -> (7,15) -> (5,32) -> (9,36) -> (1,16) -> (6,23) ->
(7,14) -> (4,12) -> (7,36) -> (4,23) -> (5,10) -> (6,26) -> (5,5) ->
(5,20) -> (1,9) -> NULL**

Após aplicar o algoritmo descrito acima, qual é a soma dos *timestamps* das mensagens que permanecem na lista?

R: 250

2. Uma empresa de software desenvolveu um sistema de apoio ao cliente baseado em tickets. Um ticket é definido pela seguinte estrutura.

```
struct ticket {  
    int arrivalTime;  
    int timeToSolve;  
    int premiumCustomer;  
}
```

O *arrivalTime* corresponde ao instante de chegada, em minutos, do ticket ao sistema, e o *timeToSolve* ao tempo necessário, em minutos, para resolver o ticket.

Os tickets são, normalmente, atendidos por ordem de chegada (*First In First Out*), um de cada vez. No entanto, clientes premium têm prioridade no atendimento, isto é, se houver um pedido de um cliente premium por atender, este deve ser atendido antes dos pedidos dos restantes clientes "não premium". Entre clientes premium, a prioridade de atendimento é por ordem de chegada.

Uma métrica importante para avaliar a satisfação dos clientes é o tempo de serviço de um ticket, isto é, quanto tempo passou desde que o ticket entrou no sistema até este ter sido resolvido. Por exemplo, considere o array:

```
struct ticket tickets[] = { {0, 5, 0}, {2, 4, 0}, {3, 2, 1}, {4, 1, 0}, {6, 1, 1}};
```

Neste caso,

- 1º ticket é atendido de imediato, com tempo de serviço 5min.
- De seguida, atende-se o 3º _ticket_, por ser de um cliente _premium_. Este espera 2min na fila, resultando num tempo de serviço igual a 4min.
- O 5º _ticket_ é atendido de seguida, tendo esperado 1min na fila. O seu tempo de serviço é 2min.
- Não havendo clientes _premium_ à espera, o 2º _ticket_ é atendido após esperar 6min na fila. O seu tempo de serviço é 10min.
- Por fim, o 4º ticket é atendido. Tendo esperado 8min na fila, o seu tempo de serviço é 9min.

Para o seguinte conjunto de 50 tickets, indique qual o tempo de serviço máximo. Para o exemplo anterior, a resposta seria 10.

```
struct ticket tickets[50] = { {1, 1, 1}, {2, 2, 0}, {3, 2, 0}, {4, 1, 1}, {5, 1, 0},  
    {6, 5, 0}, {9, 1, 0}, {12, 2, 0}, {15, 6, 0}, {18, 5, 0}, {20, 2, 0}, {22, 1, 0},  
    {23, 6, 1}, {25, 3, 0}, {27, 2, 1}, {28, 3, 0}, {29, 1, 1}, {31, 1, 0}, {33, 3, 1},  
    {36, 3, 0}, {48, 5, 0}, {59, 1, 1}, {74, 5, 1}, {88, 2, 0}, {93, 6, 0}, {99, 2, 0},  
    {110, 3, 1}, {125, 3, 0}, {135, 3, 0}, {150, 3, 0}, {164, 6, 0}, {177, 6, 0},  
    {179, 3, 1}, {180, 3, 1}, {181, 2, 1}, {182, 3, 1}, {184, 3, 1}, {185, 2, 1},  
    {187, 2, 1}, {189, 4, 1}, {190, 2, 1}, {191, 2, 0}, {192, 3, 1}, {193, 4, 1},  
    {194, 2, 0}, {195, 4, 0}, {196, 1, 0}, {197, 2, 0}, {208, 5, 0}, {219, 4, 1}};
```

Nota: o array está ordenado por ordem de chegada. Responda apenas com um inteiro, sem unidades (ex: 10).

R: 32